

# Data Preparation and Feature Engineering for Applied AI

Missing data • Outliers • Scaling • Encoding • Leakage • Domain features

Applied AI Design Workshop for Faculty  
Fred Livingston (fjliving@ncsu.edu) 2026

---

## Goal

turn raw disciplinary data into a trustworthy modeling table.

## Session outputs

- Data-readiness checklist
- Preprocessing plan
- Leakage-risk decision
- Candidate feature list
- Minimal coding pattern

## Core idea

Model quality is bounded by data quality and representation. Better features often beat bigger models.

# Activity roadmap



**Data preparation is the bridge between problem framing and model training.**

# Why data preparation matters

Applied AI fails when the data table does not match the real decision context.

## Raw data

- Different units
- Missing values
- Text labels
- Sensor noise
- Time stamps
- Batch identifiers



## Modeling table

- One row per observation
- Clean inputs
- Target separated
- Encoded categories
- Scaled numeric fields
- No leakage



## Defensible model

- Baseline comparison
- Valid split
- Interpretable metrics
- Reproducible decisions
- Clear limitations

**Preprocessing is not clerical cleanup — it is part of the research design.**

# Part 1: audit the data before modeling

A good audit asks whether the dataset can support the intended prediction task.

| Question                                                          | Why it matters                                                  |
|-------------------------------------------------------------------|-----------------------------------------------------------------|
| What does one row represent?                                      | Defines the unit of observation and prevents mixed cases        |
| Which column is the target?                                       | Separates prediction objective from inputs                      |
| Which variables are numeric, categorical, datetime, text, or IDs? | Determines preprocessing and modeling choices                   |
| Are values missing?                                               | May require imputation, removal, or redesign                    |
| Are categories consistent?                                        | Avoids treating “A”, “a”, and “Batch A” as different categories |
| Are there outliers or impossible values?                          | May indicate errors, rare events, or valid extremes             |

A good data audit reduces the risk of training a high-scoring but meaningless model.

## Part 2: clean missing values and outliers carefully

Missingness and extreme values can be errors, process signals, or meaningful rare events.

Missing values

- Common actions:
- Remove only when justified
  - Impute median or mode
  - Add a missingness indicator
  - Revisit data collection

Outliers

- Possible actions:
- Correct impossible values
  - Preserve rare valid cases
  - Cap or transform values
  - Use robust models

Key caution

Fit imputation and scaling rules only on training data, then apply them to validation/test data.

| Example                                          | Interpretation                       | Decision                                    |
|--------------------------------------------------|--------------------------------------|---------------------------------------------|
| Vibration missing during high-speed runs         | Sensor saturation may be informative | Add missingness flag and investigate sensor |
| pH missing at random                             | Small random gaps                    | Median imputation may be acceptable         |
| Final inspection score missing before inspection | Prediction-time leakage risk         | Do not use as input                         |

# Part 3: choose transformations by data type

Models need numeric arrays, but disciplinary data often contain categories, IDs, dates, and text.

| Data type   | Examples                                   | Typical handling                                    |
|-------------|--------------------------------------------|-----------------------------------------------------|
| Numeric     | temperature, pressure, speed               | Impute missing values; scale if needed              |
| Categorical | material type, machine ID, treatment group | One-hot encode or ordinal encode carefully          |
| Datetime    | timestamp, collection date                 | Extract elapsed time, hour, day, cycle, or lags     |
| Identifier  | part ID, student ID, batch ID              | Usually remove; use for grouping or split strategy  |
| Text        | notes, descriptions, comments              | Use separate NLP workflow; avoid accidental leakage |

Question to ask: is this variable a meaningful predictor, an identifier, or a disguised answer?

## Part 4: scale numeric features and encode categories

Transformations should make inputs usable without changing the meaning of the problem.

### Scaling

Why it matters:  
variables measured in large units can dominate distance-based or coefficient-based models.

Use for KNN, SVM, PCA, logistic regression, and regularized regression.

### Encoding

Why it matters:  
models cannot use raw text categories.

Use one-hot encoding for categories without natural order. Be careful with ordinal encoding.

### Pipeline rule

Fit scalers, encoders, and imputers on training data only. Apply the learned transformations to validation and test sets.

**Preprocessing rules are learned from the training set, not from the full dataset.**

## Part 5: data leakage can invalidate a model

Leakage creates inflated performance by letting the model see information unavailable in real use.

### Target leakage

Input contains the answer or a near-answer.

Example: final inspection score used to predict pass/fail quality.

### Time leakage

Future information is used to predict an earlier outcome.

Example: using post-failure data to predict pre-failure status.

### Group leakage

Related samples appear in both train and test sets.

Example: parts from the same batch split across train and test.

### Self Activity

For each feature, ask: would this value be known at the moment the prediction is needed?

## Part 6: engineer features with disciplinary meaning

Feature engineering converts raw measurements into variables that better represent mechanisms, behavior, or context.

| Raw data                | Engineered feature                                  | Why useful                                   |
|-------------------------|-----------------------------------------------------|----------------------------------------------|
| Temperature time series | slope, max, time above threshold                    | Captures trend and thermal exposure          |
| Sensor signal           | rolling mean, variance, peak count                  | Summarizes stability and abnormal variation  |
| Process settings        | distance from nominal setting                       | Represents deviation from intended operation |
| Student activity logs   | completion rate, delay, weekly activity             | Summarizes engagement over time              |
| Bioprocess trajectories | area under curve, growth rate, final-initial change | Captures dynamic process behavior            |

Good features make the model inputs closer to how a domain expert reasons about the system.

# Part 7: select features to simplify the model

More variables are not always better. Selection can improve interpretation, cost, and robustness.

## Filter methods

Use correlation, variance, or statistical association before training a model.

## Embedded methods

Use model structure such as Lasso coefficients or tree-based importance.

## Domain selection

Keep variables that are reliable, available early, and meaningful to collaborators.

## Selection criteria

Predictive value

Interpretability

Measurement cost

Available at prediction time

Robust across groups

Feature selection is a design decision, not only a statistical procedure.

## Session 3 technical skill: pandas data audit

Goal: quickly inspect structure, missingness, variable types, and target balance.

```
import pandas as pd

df = pd.read_csv("process_data.csv")

print(df.shape)
display(df.head())

summary = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing": df.isna().sum(),
    "unique": df.nunique()
})

display(summary)

# For classification targets
display(df["defect"].value_counts(normalize=True))
```

### Use before modeling

This audit should happen before selecting an algorithm.

### prompt

“What would stop us from trusting a model trained on this table?”

Session 3 notebook

03\_session3\_data\_preparation\_feature\_engineering.ipynb

## Session 3 technical skill: safe preprocessing pipeline

Goal: fit preprocessing decisions on training data only and apply them consistently to test data.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_features = ["temperature", "pressure", "vibration"]
categorical_features = ["material_type"]

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"),
     categorical_features)
])

# Later: Pipeline([("prep", preprocess), ("model", model)])
```

### Why pipeline?

It prevents train/test contamination and documents the workflow.

### Key principle

Never use test data to decide imputation, scaling, or encoding rules.

### Link to baseline modeling

Use the preprocessing object inside the baseline model workflow.

## Session 3 wrap-up: what you should take forward

### Before modeling

- Define the unit of observation
- Identify target and inputs
- Audit data types
- Check missing values
- Check leakage risk

### During preprocessing

- Fit transformations on training data
- Impute carefully
- Scale when needed
- Encode categories
- Preserve meaningful outliers

### After preprocessing

- Document decisions
- Explain limitations
- Revisit data collection
- Engineer better features
- Prepare for model comparison

**Next: Unsupervised Learning for Discovery — exploring structure when target labels are unavailable.**